

WEEK 6

Task -1: Create a client-server application using Named Pipes (FIFOs). The client sends messages and the Server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

CODE:

```
GNU nano 8.7.1 server.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <unistd.h>

#define FIFO1 "client_to_server"
#define FIFO2 "server_to_client"

int main() {
    char message[100];
    char response[100];

    mkfifo(FIFO1, 0666);
    mkfifo(FIFO2, 0666);

    printf("Server started...\n");

    int fd1 = open(FIFO1, O_RDONLY);
    int fd2 = open(FIFO2, O_WRONLY);

    while (1) {
        read(fd1, message, sizeof(message));

        printf("Client: %s\n", message);

        if (strcmp(message, "exit") == 0)
            break;

        sprintf(response, "Server received: %s", message);

        write(fd2, response, strlen(response) + 1);
    }

    close(fd1);
    close(fd2);

    unlink(FIFO1);
    unlink(FIFO2);

    return 0;
}
```

CODE:

```
pavani@DESKTOP-38OLEIN: ~/fifo_project
GNU nano 8.7.1 client.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>

#define FIFO1 "client_to_server"
#define FIFO2 "server_to_client"

int main() {
    char message[100];
    char response[100];

    int fd1 = open(FIFO1, O_WRONLY);
    int fd2 = open(FIFO2, O_RDONLY);

    while (1) {
        printf("Enter message: ");
        fgets(message, sizeof(message), stdin);

        message[strcspn(message, "\n")] = '\0';

        write(fd1, message, strlen(message) + 1);

        if (strcmp(message, "exit") == 0)
            break;

        read(fd2, response, sizeof(response));

        printf("Server: %s\n", response);
    }

    close(fd1);
    close(fd2);

    return 0;
}
```

OUTPUT:

```
pavani@DESKTOP-380LEIN:~/fifo_project$ gcc client.c -o client
pavani@DESKTOP-380LEIN:~/fifo_project$ ls
client client.c server server.c
pavani@DESKTOP-380LEIN:~/fifo_project$ ./server
Server started...
Client: Hello Server
Client: exit
Server stopped.
pavani@DESKTOP-380LEIN:~/fifo_project$ █
```

Report: The client-server application was successfully implemented using Named Pipes (FIFOs) in Ubuntu.

The client sends messages through the FIFO, and the server receives and processes them.

The server then sends a response back to the client through another FIFO.

The communication was tested successfully, and FIFO behavior with multiple clients was analyzed.

Task-2Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers

Code:

```
GNU nano 8.7.1                                signal.c *
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handler(int sig)
{
    if (sig == SIGINT)
        printf("\nSIGINT received!\n");

    else if (sig == SIGTERM)
        printf("\nSIGTERM received!\n");

    else if (sig == SIGUSR1)
        printf("\nSIGUSR1 received!\n");
}

int main()
{
    signal(SIGINT, handler);
    signal(SIGTERM, handler);
    signal(SIGUSR1, handler);

    printf("Process ID: %d\n", getpid());
    printf("Waiting for signals...\n");

    while (1)
    {
        printf("Program is running...\n");
        sleep(3);
    }

    return 0;
}
```

OUTPUT:

```
pavani@DESKTOP-380LEIN:~$ mkdir signal_project
signal_projectpavani@DESKTOP-380LEIN:~$ cd signal_project
pavani@DESKTOP-380LEIN:~/signal_project$ nano signal.c
pavani@DESKTOP-380LEIN:~/signal_project$ gcc signal.c -o signal
pavani@DESKTOP-380LEIN:~/signal_project$ ./signal
Process ID: 1385
Waiting for signals...
Program is running...
Program is running...
^C
SIGINT received!
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
SIGUSR1 received!
Program is running...
```

REPORT: The POSIX signal handling program was successfully implemented in Ubuntu.

The program handles SIGINT, SIGTERM, and SIGUSR1 using signal handlers.

Signals were generated using Ctrl+C and the kill command from another terminal.

The program responded to the signals asynchronously while continuing its normal execution.